

HACKATHON GUIDE

Platform Architecture, Customization Directory & Evaluator Cheat Sheet

This is your cheat sheet for the demo. Every answer below points to a real file — open it and you'll see exactly what it says here.

IF A JUDGE ASKS YOU TO CHANGE...

WEBSITE & FRONTEND

Website / platform title & hero headline

file: **frontend/index.html**

- `<title>` tag near the top (~line 8)
- Hero headline: `<h1>Turn Competency Gaps Into...</h1>` (~line 50)
- Subtitle & stats counters: lines ~51–60

Landing page sections (About, How It Works, Features)

file: **frontend/index.html**

- About section: `#about` (~line 79)
- 7-stage capability cycle: `#process-rail` (~line 111)
- Feature cards: `#features` (~line 124)

Frontend styling, colors & visual theme

file: **frontend/css/style.css**

- CSS Variables & theme tokens at top (`:root` → `--signal` , `--indigo` , `--amber` , `--bg` , etc.)
- AI Assessment styling: search for `.ai-mode-grid` , `.ai-mode-card` , `.upload-zone` , `.ai-status-box`

Sidebar navigation items & breadcrumbs

- Navigation buttons: `frontend/index.html` inside `<nav id="learner-nav">` and `<nav id="admin-nav">`
- Breadcrumb titles: `frontend/js/router.js` → `BREADCRUMBS` map
- Route registration: `frontend/js/app.js` → `setRenderers()`

Gated view permissions (which pages require diagnostic test first)

file: **frontend/js/router.js**

- variable: `GATED_VIEWS` (e.g. `dashboard` , `competencies` , `learning-path` , `igot` , `progress`)
- (Note: `ai-assessment` is un-gated so judges and officers can test AI assessments immediately!)

AI ASSESSMENT & RAG PIPELINE (MAIN FEATURE)

AI Assessment user interface (Dual-Mode)

file: `frontend/js/pages/ai.js`

→ Mode 1 (PDF Upload / RAG from knowledge base or direct upload): `renderPdfMode()`

→ Mode 2 ("According to You" personalized gap assessment): `renderPersonalMode()`

→ API triggers: `startPdfAssessment()` and `startPersonalAssessment()`

Local NLP Question Generation (Extractive RAG without API key)

file: `backend/services/nlpQuestionGenerator.js`

→ Question pattern templates (definitions, functions, components): `synthesizeQuestionFromSentence()`

→ Sentence splitting & cleaning (removes bullets, formats text): `splitSentences()` , `cleanText()`

→ Distractor options & pool: `generateDistractors()` (matches grammar and extracts clauses from document text)

AI Question Generator Orchestrator (LLM with NLP fallback)

file: `backend/services/questionGenerationService.js`

→ `generateQuestionsFromDocument()` : retrieves top-K chunks via TF-IDF, calls LLM or local NLP

→ `generateLiveQuestionSet()` : handles personalized gap questions from knowledge base

AI assessment API routes

file: `backend/routes/assessments.js`

→ `POST /api/assessments/ai-from-document` (RAG from uploaded/selected PDF)

→ `POST /api/assessments/ai-personalized` ("According to You" gap-targeted)

→ `GET /api/assessments/available-documents` (lists ready PDFs in Knowledge Base)

RAG document extraction, chunking & vector search

→ Text extraction from PDF: `backend/services/documentService.js` (uses Mozilla `pdfjs-dist`)

→ Chunking & TF-IDF vectors: `backend/services/embeddingService.js` (`chunkText()` , `buildTfidfVectors()` , `findTopK()`)

→ Chunk size & overlap: `backend/config/appConfig.js` (`CHUNK_SIZE_CHARS` , `CHUNK_OVERLAP_CHARS` , `RETRIEVAL_TOP_K`)

QUESTION BANK & SCORING

Hand-authored diagnostic question bank (questions, options, correct answers)

file: `data/demoData.js` → `QUESTION_BANK` array (50+ verified MCQs covering all 10 competencies)

→ Format:

```
{
  q: 'Question text here?',
  comp: 'Sampling',
  sub: 'Sampling Methods',
  diff: 'Medium', // Easy | Medium | Hard
  opts: ['Option A', 'Option B', 'Option C', 'Option D'],
  correct: 1, // 0-based index of correct option
  explanation: 'Why Option B is correct.'
}
```

→ To reload updated questions into SQLite: run `node backend/database/seed.js --force`

Number of diagnostic assessment questions

file: `backend/config/appConfig.js`

→ variable: `NUMBER_OF_ASSESSMENT_QUESTIONS` (default: 10)

Passing score cutoff

file: `backend/config/appConfig.js`

→ variable: `PASSING_SCORE` (default: 70)

Assessment scoring logic & adaptive difficulty rules

file: `backend/services/assessmentService.js`

→ Scoring: `calculateScore()` , `calculateCompetencyScores()`

→ Rule-based adaptive difficulty: `nextDifficulty()` (correct moves up, incorrect moves down)



COMPETENCIES, ROLES & LEARNING

Add or edit competencies & required scores

file: `data/competencies.js`

→ `COMPETENCY_LIB` dictionary:

```
'New Competency': { required: 80, sub: ['Sub A', 'Sub B'] }
```

→ Map it to roles in `data/roles.js` (e.g. `collector` , `field` , `analyst`)

→ Re-seed: `node backend/database/seed.js --force`

Role definitions & competency mapping

file: `data/roles.js` → `ROLES` array

Competency gap thresholds (HIGH / MEDIUM / LOW cutoffs)

file: `backend/config/appConfig.js`

→ variables: `HIGH_GAP_THRESHOLD` , `MEDIUM_GAP_THRESHOLD`

→ Gap calculation: `backend/services/assessmentService.js` → `classifyGap()`

iGOT Karmayogi course recommendations

→ Course catalogue: `data/courses.js`

→ Recommendation matching engine: `backend/services/igotService.js` (`matchCoursesToGaps()`)

USERS, PASSWORDS & CONFIG

Demo officer accounts & credentials

file: `data/demoData.js` → OFFICERS array

→ Default demo password for all accounts: `"demo1234"` (hashed in `backend/database/seed.js`)

→ Admin account: `admin@mospi.gov.in` / `demo1234`

→ Learner account: `officer.sharma@mospi.gov.in` / `demo1234`

Server port & external LLM API key

→ Port: `backend/config/appConfig.js` → PORT (default: 4000)

→ Anthropic API key (optional, for Claude LLM generation): `.env` → `ANTHROPIC_API_KEY=`

→ (If no key is set, the system automatically runs the local RAG NLP extractor!)

WHY SQLite INSTEAD OF POSTGRESQL?

The brief asked for PostgreSQL. We used SQLite instead, on purpose, because:

- No separate database server to install, start, or configure — one less thing to go wrong live on stage
- The whole database is a single file (`backend/database/data.sqlite`) — `npm install && npm start` and you're running
- It is still a **real** relational database: real SQL, real foreign keys, real persisted rows — nothing about the data layer is simulated

If you want to switch to Postgres later: `backend/database/schema.sql` is plain SQL. Swap `AUTOINCREMENT` → `SERIAL` , point `backend/database/db.js` at a `pg` client instead of `better-sqlite3` , and the rest of the app (all the `db.prepare(...).run().get().all()` calls) would need a small adapter layer, but the schema itself transfers directly.

LIKELY JUDGE QUESTIONS — SIMPLE ANSWERS

"Is this actually using a real database, or is it still mock data?"

Real SQLite database, real SQL tables, real foreign keys. `backend/database/schema.sql` has the full schema. Every assessment, score, and document you see was written by a real INSERT statement, not a JavaScript array.

"Where does the frontend talk to the backend?"

One file: `frontend/js/api.js`. Every network call in the whole app goes through it.

"Where is the assessment actually scored?"

`backend/services/assessmentService.js`. The frontend sends which option was clicked; the backend looks up the correct answer and computes the score. The frontend never sends a score.

"Is the RAG pipeline real or simulated?"

Real! The entire pipeline: PDF upload → text extraction via Mozilla `pdfjs-dist` → sliding window chunking → TF-IDF vectorization → cosine similarity retrieval all run live on real data (see `backend/services/documentService.js` and `embeddingService.js`).

For question generation:

- If `ANTHROPIC_API_KEY` is set in `.env`: it calls Claude 3.5 Sonnet to craft questions grounded in the retrieved chunks.
- If no API key is set: it runs our **Local NLP Extractor** (`backend/services/nlpQuestionGenerator.js`), which performs semantic sentence parsing, grammatical relation extraction, and generates distractors and verifiable correct answers directly from the document chunks.

"How does the dual-mode AI assessment work?"

There are two distinct modes accessible from the AI Assessments page (`frontend/js/pages/ai.js`):

1. **Upload PDF Document (RAG)**: The officer uploads any PDF or picks an existing manual from the Knowledge Base. The RAG pipeline indexes the file, retrieves top chunks for the chosen topic and difficulty, and generates questions grounded in that text.
2. **"According to You" (Personalized)**: The engine inspects the officer's competency profile, identifies their largest competency gap, retrieves relevant material from indexed documents, and generates targeted questions specifically to close that gap.

"Why don't I see my competency scores right after logging in?"

By design. A brand-new officer has no real assessment data yet, so showing scores would mean showing fake numbers. The app sends a first-time officer straight to the diagnostic assessment (`frontend/js/app.js` → `enterApp()`), and gated pages (dashboard, competencies, learning path, iGOT, progress) redirect back there until `hasCompletedDiagnostic` is true — see `GATED_VIEWS` in `frontend/js/router.js`. (AI Assessments is accessible anytime so officers can test document-based RAG right away).

"Is the iGOT integration real?"

No — clearly labeled demo/prototype data (`data/courses.js`, `backend/services/igotService.js`). No real iGOT Karmayogi API credentials were available for this build. What IS real: the matching logic — `GET /api/igot/recommendations` finds the officer's actual weakest competency + sub-competency and returns the course tagged for that exact combination first, with a `matchReason` explaining why.

"Is the login real government authentication?"

No — demo login against our own database (bcrypt-hashed passwords, JWT sessions). Labeled "DEMO_LOGIN" in the API response. Not a Government Identity Provider integration.

"How do you know a generated question really came from the document?"

Every RAG-generated question stores `source_document`, `source_page`, and the exact `source_chunk_id` it was grounded in (see the `questions` table in `schema.sql`). The admin review screen shows the retrieved excerpt next to the question before you approve it — you can open the PDF and check it yourself.

"What happens if the backend or AI API is down?"

The frontend shows a plain-language error via `friendlyError()` (see `frontend/js/utils.js`) instead of crashing. AI features fall back to clearly-labeled demo responses instead of hanging or pretending to work — see `backend/services/llmService.js`.

"Can a regular officer see admin data?"

No — every admin route is protected by `requireAdmin` middleware (`backend/middleware/auth.js`), checked against the officer's role stored in their signed JWT, not anything the frontend claims. The "Switch to Admin View" button is even hidden in the UI for non-admins.

KNOWN LIMITATIONS (say these proactively, it builds trust)

- PDF is the only supported document format (by design — see `SUPPORTED_FILE_TYPES` in `appConfig.js`)
- PDF export for reports isn't implemented — CSV is, and is real. This was a deliberate scope cut (see honest status table in README.md)
- Retrieval uses TF-IDF, not neural embeddings (explained above)
- No real government identity provider or iGOT Karmayogi integration
- The reassessment/AI-assessment flows don't currently support navigating back to a previous question mid-assessment (kept simple on purpose)
- If the hand-authored question bank has fewer distinct questions than `REASSESSMENT_QUESTION_COUNT` for a given gap, the reassessment attempt is honestly shorter rather than repeating a question to pad the count (see `shortSet` flag in the API response)